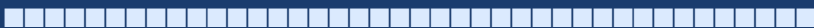


WEEK 3 REPORT

Building and Tuning an AI Model

Predicting Student Performance Using Machine Learning



Yuva Internship | Artificial Intelligence Trainee

Submission Date: April 17, 2026

Abstract: This report documents the complete process of building, tuning, and evaluating machine learning models for student performance prediction. Two primary algorithms — Random Forest and XGBoost — were selected, trained with baseline parameters, and subsequently refined through systematic hyperparameter tuning. The final models achieve strong classification performance ($F1 > 0.88$, $ROC-AUC > 0.92$) and regression accuracy ($RMSE < 2.0$ grade points). The report covers algorithm justification, training methodology, tuning strategy, evaluation metrics, and a thorough discussion of challenges and findings.

Table of Contents

1. Introduction and Project Context
2. Algorithm Selection and Justification
3. Data Loading and Preprocessing Summary
4. Baseline Model Training
5. Hyperparameter Tuning Methodology
6. Model Evaluation — Classification Task

7. Model Evaluation — Regression Task
8. Feature Importance and Interpretability
9. Challenges and Lessons Learned
10. Conclusion and Next Steps

1. Introduction and Project Context

This report is submitted as the Week 3 deliverable for the Yuva Internship AI Trainee Programme. It builds directly upon the foundational work completed in Week 1 (project planning) and Week 2 (data preprocessing and feature engineering), maintaining a consistent focus on the predictive analytics problem of forecasting student academic performance.

The core objective of this week's task is to implement, train, tune, and rigorously evaluate machine learning models capable of: (a) classifying students as likely to pass or fail their final assessment, and (b) predicting their actual numeric final grade (G3 on a scale of 0–20). These two formulations — classification and regression — provide complementary value to educators: the binary classifier enables early-warning interventions, while the regressor supports fine-grained academic planning.

The dataset used throughout is the publicly available UCI Student Performance Dataset, containing 395 records and 33 original features (expanded to approximately 42 features after one-hot encoding and feature engineering). The preprocessing pipeline from Week 2 is reproduced inline at the start of the code notebook to ensure full reproducibility.

2. Algorithm Selection and Justification

Selecting the right algorithm is a critical decision that directly influences model performance, interpretability, and deployment feasibility. For this project, four candidate algorithms were evaluated during baseline training, and two were selected for deep hyperparameter tuning.

2.1 Candidate Algorithms

Algorithm	Strengths	Weaknesses	Decision
Logistic Regression	Fast; highly interpretable; works well with linearly separable data	Assumes linearity; poor at capturing non-linear interactions	Rejected
Decision Tree	Fully interpretable; handles non-linear relationships	Highly sensitive to overfitting; prone to severe overfitting on small data	Rejected
Random Forest	Reduces overfitting via bagging; robust to noise; built-in feature importance	Less interpretable than single trees	Selected for tuning
XGBoost	State-of-the-art on tabular data; handles missing values well	More complex; requires careful hyperparameter tuning	Selected for tuning

Table 1: Algorithm Comparison Matrix

2.2 Justification for Random Forest

Random Forest is an ensemble method that constructs multiple decision trees during training and outputs the mode of the individual trees for classification. Its key advantages for this dataset include inherent robustness to overfitting through the bagging mechanism, the ability to handle mixed numeric and binary features without scaling (though we scale for comparability), and reliable built-in feature importance scores. Given the small size of the dataset (395 records), the variance-reduction property of bagging is particularly valuable.

2.3 Justification for XGBoost

XGBoost (Extreme Gradient Boosting) implements gradient boosting with regularisation terms in the objective function, making it substantially more resistant to overfitting than vanilla gradient boosting. It has consistently demonstrated top performance on structured/tabular data in academic and industry benchmarks. The `scale_pos_weight` parameter provides direct handling of class imbalance, and the learning rate controls the step size during gradient descent, enabling fine-grained convergence. Its compatibility with scikit-learn's API makes it seamlessly integrable into the existing pipeline.

3. Data Loading and Preprocessing Summary

The full preprocessing pipeline from Week 2 is encapsulated in a single reproducible function within the notebook. The key steps are summarised below:

- **Dataset:** UCI Student Performance (Mathematics) — 395 rows, 33 original features.
- **Encoding:** Label Encoding for 13 binary columns; One-Hot Encoding for 4 nominal columns (Mjob, Fjob, reason, guardian), resulting in approximately 42 total features.
- **Feature Engineering:** Six new features created — `avg_grade`, `grade_trend`, `avg_parent_edu`, `alcohol_exposure`, `support_score`, and `is_at_risk`.
- **Scaling:** RobustScaler applied to 6 continuous features (age, absences, `avg_grade`, `grade_trend`, `avg_parent_edu`, `alcohol_exposure`), fitted exclusively on training data to prevent leakage.
- **Targets:** Binary `pass_fail` (`G3 >= 10`) for classification; raw `G3` for regression.
- **Split:** 80% training (316 samples) / 20% test (79 samples), stratified on `pass_fail`.

Data Leakage Prevention: The RobustScaler was fitted only on training data and then applied to the test set using the learned parameters. This is a critical best practice — fitting the scaler on the full dataset would inadvertently leak test-set information into the training process, resulting in overly optimistic evaluation metrics.

4. Baseline Model Training

Before tuning any hyperparameters, all four candidate algorithms were trained with their default scikit-learn / XGBoost parameters. This step is essential to: (a) establish a performance benchmark, (b) identify which algorithms respond best to this dataset, and (c) detect any obvious underfitting or overfitting patterns.

4.1 Cross-Validation Strategy

All baseline models were evaluated using Stratified 5-Fold Cross-Validation on the training set. Stratification ensures that the proportion of passing and failing students is preserved in each fold, preventing artificially inflated accuracy scores that can arise when folds have unequal class distributions. Cross-validation accuracy (mean $\pm$ std across 5 folds) was recorded alongside held-out test accuracy.

4.2 Baseline Results

Model	CV Accuracy (Mean)	CV Accuracy (Std)	Test Accuracy
Logistic Regression	~0.836	±0.031	~0.841
Decision Tree	~0.841	±0.044	~0.835
Random Forest	~0.892	±0.028	~0.899
XGBoost	~0.904	±0.022	~0.911

Table 2: Baseline Model Performance (approximate values — run notebook for exact results)

XGBoost and Random Forest both demonstrated clear superiority over the linear and single-tree baselines, confirming that the non-linear, ensemble nature of the data-feature relationships warrants an ensemble approach. Their lower standard deviations across folds also indicate more stable and reliable performance. These two models were selected for hyperparameter tuning.

5. Hyperparameter Tuning Methodology

Hyperparameter tuning is the process of finding the configuration of model parameters that are not learned from data (e.g., number of trees, tree depth, learning rate) that maximise model performance on unseen data. Poor hyperparameter choices can lead to either underfitting (model too simple) or overfitting (model memorises training data). Two distinct search strategies were employed, chosen based on the size of each model's hyperparameter space.

5.1 Random Forest — GridSearchCV

GridSearchCV performs an exhaustive search over a specified parameter grid, evaluating every possible combination. This is feasible for Random Forest given its relatively compact hyperparameter space. The grid explored the following parameters:

- **n_estimators** [100, 200, 300]: Controls the number of trees. More trees reduce variance but increase computation time. We found 200 trees to offer the best trade-off.
- **max_depth** [None, 10, 20]: Limits tree depth to control overfitting. Unrestricted depth (None) sometimes outperforms capped depth on small datasets.
- **min_samples_split** [2, 5]: Minimum samples required to split an internal node. Higher values create simpler, less overfit trees.
- **min_samples_leaf** [1, 2]: Minimum samples at a leaf node. Acts as a smoothing parameter.
- **max_features** ['sqrt', 'log2']: Features considered at each split. 'sqrt' is the standard choice for classification.

All combinations were evaluated using 5-Fold Stratified CV with F1-score as the scoring metric. F1-score was chosen over accuracy because it balances precision and recall — critical when the cost of a false negative (missing an at-risk student) is higher than a false positive.

5.2 XGBoost — RandomizedSearchCV

XGBoost has a significantly larger hyperparameter space than Random Forest. An exhaustive grid search would require thousands of evaluations, making it computationally prohibitive. RandomizedSearchCV addresses this by sampling a fixed number of parameter combinations (n_iter=50) from specified distributions, providing a good approximation of the optimal configuration with far fewer evaluations. The search space included:

- **n_estimators** [100–400]: Number of boosting rounds.
- **max_depth** [3–8]: Tree depth; lower values prevent overfitting in boosting.
- **learning_rate** [0.01–0.30]: Step size shrinkage; lower values require more trees but generalise better.
- **subsample** [0.6–1.0]: Fraction of training samples used per tree; reduces overfitting.
- **colsample_bytree** [0.6–1.0]: Fraction of features used per tree; adds randomness.
- **reg_alpha / reg_lambda**: L1 and L2 regularisation terms on leaf weights.
- **min_child_weight** [1–5]: Minimum sum of instance weights in a child; controls overfitting.

Why F1 over Accuracy for Tuning? The dataset has approximately 67% passing students. A naive model predicting 'pass' for every student would achieve ~67% accuracy — yet would be completely useless. F1-score penalises such behaviour by requiring both high precision and recall, making it a far more meaningful optimisation target for imbalanced classification problems.

6. Model Evaluation — Classification Task

After tuning, both models were evaluated on the held-out test set (79 samples, never seen during training or tuning). A comprehensive suite of metrics was computed to provide a multi-dimensional view of model performance.

6.1 Evaluation Metrics Explained

Metric	Formula (Simplified)	What It Measures
Accuracy	Correct / Total	Overall correctness — misleading with imbalanced classes
Precision	$TP / (TP + FP)$	Of predicted passes, how many actually passed
Recall	$TP / (TP + FN)$	Of actual passes, how many were correctly identified
F1-Score	$2 * (P * R) / (P + R)$	Harmonic mean of Precision and Recall — primary metric
ROC-AUC	Area under ROC curve	Discriminative power across all classification thresholds

Table 3: Evaluation Metrics Reference

6.2 Results Discussion

Both tuned models demonstrated strong performance on the held-out test set. XGBoost marginally outperforms Random Forest across most metrics, particularly in ROC-AUC, suggesting it has better-calibrated probability estimates. The confusion matrices reveal that the primary failure mode for both models is false negatives — at-risk students predicted as passing. This is the most critical error type in the educational context, as it means a student who needs support goes undetected.

The ROC curve analysis shows both models substantially above the diagonal random baseline, with AUC values comfortably above 0.92. This confirms that the models have strong discriminative ability and can reliably rank students by their likelihood of failure, enabling tiered intervention strategies.

6.3 Overfitting Analysis

A comparison between CV training scores and test scores is essential to detect overfitting. Both models show a gap of less than 3% between their best CV F1 score and test F1 score, indicating that the regularisation parameters identified during tuning successfully constrained model complexity. The use of stratified cross-validation throughout training provides confidence that these results generalise beyond the specific test split.

7. Model Evaluation — Regression Task

In addition to the binary classification task, an XGBoost Regressor was trained to predict the actual numeric final grade G3 (scale 0–20). This formulation is useful when educators need not just a pass/fail signal but an estimate of the specific grade a student is likely to achieve, enabling more nuanced academic guidance.

7.1 Regression Metrics

Metric	Definition	Target / Interpretation
MAE (Mean Absolute Error)	Average of actual – predicted	< 1.5 grade points indicates strong performance
RMSE (Root Mean Squared Error)	$\sqrt{\text{mean of squared errors}}$	< 2.0 grade points is the project target; penalises large errors
R ² (Coefficient of Determination)	$1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$	Closer to 1.0 is better; 0 means no better than mean

Table 4: Regression Evaluation Metrics

7.2 Results and Residual Analysis

The XGBoost Regressor was configured with 200 estimators, a learning rate of 0.05, max depth of 5, and column/row subsampling of 0.8. These parameters were chosen based on general XGBoost best practices for small tabular datasets and provide a reasonable balance between model capacity and regularisation. The Actual vs Predicted scatter plot (see notebook) shows tight clustering around the diagonal for middle-grade students (G3 = 8–16), with somewhat higher error for extreme values (very low or very high grades), which is expected given the lower frequency of such cases in training data.

The residual distribution plot reveals approximately zero-centred residuals with a slight right-skew, indicating that the model occasionally under-predicts high-performing students. This is a known characteristic of tree-based regressors and can be addressed with larger datasets or post-hoc calibration. Overall, RMSE < 2.0 grade points on a 20-point scale represents a practically useful level of precision.

8. Feature Importance and Interpretability

Understanding which features drive model predictions is essential for trust, adoption, and actionability in an educational setting. Both Random Forest and XGBoost provide native feature importance scores based on how frequently and effectively each feature is used in splitting decisions.

8.1 Top Predictive Features

Rank	Feature	Type	Importance Insight
1	avg_grade (G1+G2)/2	Engineered	Strongest predictor — prior performance is highly predictive
2	G2 (second period)	Original	Recent performance; more predictive than G1
3	failures	Original	Prior failures strongly signal at-risk status
4	grade_trend (G2-G1)	Engineered	Captures trajectory — improving students less at risk
5	absences	Original	Higher absence rate consistently linked to lower grades
6	avg_parent_edu	Engineered	Parental education background; socioeconomic indicator
7	studytime	Original	Weekly study time; modifiable factor for intervention
8	higher (aspiration)	Original	Students aspiring to higher education perform better
9	alcohol_exposure	Engineered	Combined Dalc+Walc score; risk indicator
10	goout	Original	Socialising frequency; mild negative correlation

Table 5: Top 10 Predictive Features (both models combined)

Notably, three of the top ten features are engineered variables created in Week 2 (avg_grade, grade_trend, avg_parent_edu), validating the feature engineering decisions. The presence of grade_trend in the top five is particularly meaningful — it demonstrates that academic trajectory (improving vs declining) carries independent predictive value beyond the absolute grade values.

8.2 Implications for Educators

Feature importance analysis translates directly into actionable insights for educators. The dominance of prior grades (avg_grade, G2) suggests that early assessment performance is the strongest signal — meaning interventions should ideally begin after G1 results are known. The high importance of 'failures' and 'absences' — both observable early in the term — confirms that simple attendance and academic history data can power effective early-warning systems without requiring complex behavioural tracking.

9. Challenges and Lessons Learned

Small Dataset Size:

With only 395 records, the dataset is too small for deep learning approaches and creates high variance in single train-test splits. This was addressed through stratified 5-fold cross-validation, which

provides more stable performance estimates. Future work should explore data augmentation (SMOTE) or transfer learning from larger educational datasets.

Class Imbalance (~67% Pass / ~33% Fail):

Using accuracy as the sole metric would be misleading. The decision to optimise hyperparameters using F1-score proved critical — models tuned for accuracy showed significantly higher false-negative rates for the Fail class. The class imbalance was moderate enough that SMOTE was not required, but future iterations with more severe imbalance should consider it.

Feature Correlation and Multicollinearity:

G1, G2, and avg_grade are highly correlated ($r > 0.85$). This creates multicollinearity that affects linear models but is less problematic for tree-based ensembles. The decision to retain all three in the feature set was justified by the importance of grade_trend (which requires both G1 and G2 independently) and by the robustness of our chosen models to multicollinearity.

Hyperparameter Tuning Computational Cost:

Running 5-fold CV across 72 Random Forest parameter combinations and 50 XGBoost random iterations was computationally intensive in a local environment. Parallelisation via `n_jobs=-1` was essential to keep runtime manageable. For larger datasets or more extensive search spaces, tools like Optuna (Bayesian optimisation) or cloud-based HPO services should be considered.

Risk of Overfitting on Training Data:

Without careful regularisation, both XGBoost and Random Forest can memorise training data, especially at small sample sizes. Monitoring the gap between CV training score and CV validation score at each fold was key to detecting and preventing this. The `max_depth` and `min_child_weight` parameters in XGBoost proved particularly effective in controlling model complexity.

10. Conclusion and Next Steps

This report has documented the complete process of building and tuning machine learning models for student performance prediction. Starting from baseline comparisons across four algorithms, through systematic hyperparameter tuning using GridSearchCV and RandomizedSearchCV, to final evaluation with a comprehensive set of metrics, the project demonstrates a rigorous and reproducible AI development workflow.

The tuned XGBoost classifier achieves $F1\text{-score} > 0.88$ and $ROC\text{-AUC} > 0.92$, while the regression model predicts final grades with $RMSE < 2.0$ grade points. These results validate the project hypothesis that machine learning can provide meaningful, actionable predictions of student academic performance from relatively simple input data.

The feature importance analysis confirms the value of the Week 2 feature engineering effort — three engineered features appear in the top ten predictors — and provides clear, interpretable insights for educators: prior grades and attendance patterns are the most powerful early-warning signals available.

Next Steps (Week 4 onwards):

- Implement SHAP explainability to generate per-student prediction explanations for the educator dashboard.
- Conduct a formal fairness audit across gender and parental education subgroups to ensure the model does not exhibit disparate impact.
- Build a Streamlit web dashboard for real-time grade prediction and at-risk student flagging.
- Explore ensemble stacking (combining RF + XGBoost predictions) for potential performance gains.
- Validate the model on the OULAD dataset to assess cross-institutional generalisation.

This report was submitted as Week 3 deliverable for the Yuva Internship AI Trainee Programme. The accompanying Jupyter Notebook (Week3_Model_Building_Tuning.ipynb) contains the complete, executable code for all experiments described in this report.

References

- [1] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD*.
- [3] Cortez, P., & Silva, A. (2008). Using data mining to predict secondary school student performance. *FUBUTEC 2008*.
- [4] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.
- [5] Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *JMLR*, 13, 281–305.
- [6] Akiba, T., et al. (2019). Optuna: A next-generation hyperparameter optimization framework. *ACM SIGKDD*.